The Programming Framework: A General Method for Process Analysis Using LLMs and Mermaid Visualisation

Gary Welz

Researcher, New Media Lab, CUNY Graduate Center
Email: gwelz@gc.cuny.edu
ORCID: https://orcid.org/0009-0005-7806-0892

Abstract

Scientific and technical fields rely heavily on textual process descriptions that are difficult to compare, analyse, or reuse computationally. We introduce the Programming Framework, a methodology for transforming textual process descriptions into structured, computable diagrams using large language models (LLMs) and Mermaid diagram syntax. Node colours, shapes, and edge types serve as lightweight semantic markers, providing flexible visual encoding that can be adapted to domain-specific conventions. We demonstrate the feasibility and cross-domain transferability of the methodology through application across biology (via the Genome Logic Modelling Project) and mathematics — where the framework extends naturally from algorithmic flowcharts to axiomatic dependency graphs and proof graphs, enabling unified structural comparison across procedural and logical forms. The methodology is designed to lower the barrier to process visualisation for researchers, educators, and AI systems, enabling structured comparison and analysis across scientific disciplines without requiring specialised software or domain-specific diagramming expertise. The Programming Framework is proposed as reusable, open infrastructure — a starting point that others can adopt, extend, and critique — with all methodology, tools, and examples publicly available.

Key Points

- A repeatable LLM pipeline turns textual process descriptions into structured Mermaid flowcharts across multiple scientific domains.
- A suggested five-category colour scheme—optional and customisable per domain—supports rapid visual parsing where authors adopt it; companion manuscripts may instead use palette conventions tuned to their figures (signals, activations, feedback topology).
- The Genome Logic Modelling Project (GLMP) and the Algorithms, Axiomatic Theories, and Proofs project demonstrate feasibility in two research contexts with draft papers on GitHub.
- GLMP demonstrates the framework's application to biological process visualisation—pairing LLM-assisted, diagrams-as-code flowcharts with optional reconciliation to curated regulatory resources for perturbation-oriented reasoning across regulatory networks, metabolic pathways, and cellular processes.
- The mathematics deployment (Algorithms, Axiomatic Theories, and Proofs) stores **algorithms**, **axiomatic DAGs**, and **eight-role proof graphs** (with **algorithm capsules**) in versioned JSON with public table viewers.
- Methodology, tools, and examples are open infrastructure on Hugging Face and public storage for adoption and extension.

1. Introduction

1.1 Motivation

Processes are central to scientific and technical work. Biologists describe reaction pathways and regulatory networks; computer scientists specify algorithms and data pipelines; chemists outline reaction mechanisms; physicists model physical processes; mathematicians formalise proofs and algorithms. These processes are usually documented in prose, sometimes accompanied by diagrams that are static, informal, or discipline-specific. This creates several problems:

1. Limited comparability: Two textual descriptions of similar processes may be difficult to compare systematically, even within the same discipline.

2. Low reusability: Process knowledge is not readily available as structured data for downstream tools (e.g., simulation, search, or automated checking).
3. Inconsistent notation: Disciplines use different diagramming conventions, reducing cross-domain transfer and interdisciplinary collaboration.
4. High human labor cost: Manually authoring and maintaining high-quality process diagrams is time-consuming and error-prone.

Large language models (LLMs) now provide a pragmatic way to transform text into structured representations. However, without a clear methodology and standardised representation format, attempts at "LLM-to-diagram" conversions risk being ad-hoc and irreproducible.

1.2 The Programming Framework Approach

The Programming Framework addresses these challenges by providing:

1. A suggested colour-coding system as a starting point, with flexibility for domain-specific customisation
2. A repeatable LLM-based methodology for extracting processes from text into structured flowcharts
3. Mermaid Markdown as visualisation standard for human-readable, version-controllable diagrams
4. A JSON-based storage format with metadata for managing large collections of process diagrams
5. An iterative refinement loop where humans and AI collaborate to improve process fidelity over time.

The Framework is intended as a "meta-tool": it does not prescribe what processes must look like in each field, but instead offers a consistent way to represent and refine them, enabling cross-disciplinary comparison and optimisation.

We demonstrate the feasibility and cross-domain transferability of the Programming Framework through application across multiple scientific disciplines, with the most developed deployments in biology (GLMP) and mathematics (Algorithms, Axiomatic Theories, and Proofs). The framework is proposed as infrastructure for further development, not as a validated system with formal accuracy metrics — those remain important directions for future work.

1.3 Contributions

This paper contributes:

Methodological - A repeatable LLM-based pipeline for extracting processes from text into structured flowcharts - A concrete, step-by-step methodology with prompt engineering guidelines and validation workflows - Practical observations about prompt design, failure modes, and human–AI collaboration in process modelling

Representational - A suggested five-category colour-coding system (Red/Yellow/Green/Blue/Violet) as a starting point, with examples of domain-specific customisation (e.g., GLMP's logical connectives) - Mermaid + JSON as computable, version-controlled artefacts with extensible metadata schemas

Empirical - Feasibility demonstrations in biology (Genome Logic Modelling Project) and mathematics (Algorithms, Axiomatic Theories, and Proofs), both backed by draft research papers

2. Related Work

This section situates the Programming Framework within existing research on process modelling, knowledge representation, and LLM-based diagram generation.

2.1 Process Modelling in Scientific Domains

Process modelling has a long history in scientific computing and knowledge representation. In biology, standards like SBGN (Le Novère et al., 2009), BioPAX (Demir et al., 2010), and SBML (Hucka et al., 2003) provide formal representations for biochemical networks, enabling quantitative modelling and simulation. However, these standards require specialised tools (Funahashi et al., 2008; Shannon et al., 2003) and significant expertise, creating barriers to adoption for rapid visualisation and cross-domain comparison. In computer

science, UML (Object Management Group, 2017) provides comprehensive notation for software processes, but its complexity can be overwhelming for simple process flows. Chemistry lacks a unified process visualisation standard comparable to SBGN, with tools primarily focused on molecular structure drawing rather than reaction pathways. Mathematics and physics similarly lack standardised process visualisation formats, relying on domain-specific simulation tools. The Programming Framework addresses a gap: providing accessible, text-based process visualisation that works across domains while maintaining compatibility with existing standards when needed. This work builds on principles from knowledge representation (Gruber, 1993) and ontology engineering (Noy & McGuinness, 2001), adapting them for practical, accessible process visualisation.

2.2 Knowledge Representation and Ontology Engineering

The Framework's JSON-based metadata schema draws from principles in ontology engineering and semantic web technologies. While not implementing full RDF/OWL semantics, the structured metadata approach enables entity linking, cross-referencing, and integration with knowledge graphs—core goals of semantic web research (Berners-Lee et al., 2001). The approach aligns with lightweight ontology principles (Uschold & Gruninger, 1996), prioritising practical utility over formal completeness. The colour-coding system provides a lightweight ontology for process stages (input → structure → operation → intermediate → output), enabling pattern recognition and cross-domain comparison without requiring formal ontology development. This aligns with research on visual knowledge representation (Larkin & Simon, 1987) and the use of colour semantics for information organisation (Healey & Enns, 2012).

2.3 LLM-Based Knowledge Extraction

Large language models now support structured extraction from unstructured text (Brown et al., 2020; OpenAI, 2023). The Programming Framework applies that capacity to process-specific structures—sequences, decisions, branches, and flows—using process-aware prompts and structured output methods (Ouyang et al., 2022; White et al., 2023).

2.4 Scientific Workflow Systems

Scientific workflow systems execute computational pipelines rather than documenting process logic for comparison. The Programming Framework complements them by visualising and analysing process structure; diagrams could in principle map to workflow specifications for bidirectional translation.

2.5 Diagram Generation and Visualisation

Automated diagram generation from text has been explored in various contexts (Kong et al., 2019; Liu et al., 2020). The Programming Framework's contribution is its focus on scientific processes, cross-domain applicability, and integration with version-controlled, structured data formats. The use of Mermaid Markdown provides a balance between expressiveness and accessibility, avoiding the complexity of specialised diagramming languages while maintaining sufficient structure for computational analysis.

3. Methodology

The Programming Framework treats process extraction as a structured transformation task with human-in-the-loop verification. At a high level, the pipeline has five stages:

1. Process scoping and selection
2. LLM-based structure extraction
3. Mermaid diagram generation with colour coding
4. JSON storage and metadata attachment
5. Iterative refinement (human + AI)

3.1 Process Scoping and Selection

The input to the Framework is a text source that describes a process. This can be:

1. A paragraph from a research article (e.g., a "Methods" section)
2. A textbook excerpt describing a pathway or algorithm
3. A protocol or step-by-step procedure
4. A conceptual description of a system or workflow

The scoping step consists of:

1. Identify a single process: Select a coherent process rather than mixing multiple distinct processes
2. Define the level of granularity: Decide whether the diagram should be high-level (5–10 nodes) or detailed (20+ nodes)
    - High-level (5–10 nodes): Suitable for educational overviews, conceptual understanding, initial exploration, or when the source text provides only a summary description
    - Detailed (20+ nodes): Appropriate for technical protocols, comprehensive process documentation, research methods sections, or when precise step-by-step accuracy is required
3. Specify the perspective: For example, "molecular events," "data flow," "user actions," or "computational steps"

These scoping decisions are made by a human operator but are clearly communicated to the LLM via the prompt.

3.2 Suggested Colour-Coding System

The Programming Framework suggests a five-category colour-coding system as a **practical default** for many visualisation contexts. It is **not** a definitional requirement: authors may swap in **domain- or publication-specific palettes** when those better match established figure styles or when the goal is to emphasise regulatory topology (activation, repression, feedback) rather than the five input→output roles below. Where a companion paper standardises its own colours, reusing that scheme in cross-referenced figures aids readers more than forcing a secondary mapping. This system provides a baseline semantic meaning that can be adapted for specific domains: - **Red (#ff6b6b):** Triggers & Inputs — initial conditions, environmental inputs, starting materials, or data inputs (uses white text for contrast) - **Yellow (#ffd43b):** Structures & Objects — physical structures, molecules, data structures, algorithms, or logical constructs (uses black text for readability) - **Green (#51cf66):** Processing & Operations — transformations, reactions, computations, or operations that change state (uses white text for contrast) - **Blue (#74c0fc):** Intermediates & States — intermediate products, temporary states, or transitional conditions (uses white text for contrast) - **Violet (#b197fc):** Products & Outputs — final outputs, end products, or results (uses white text for contrast)

Decision diamonds (`{...}` branch predicates) use **Blue** alongside other intermediates/states: they denote conditions or checkpoint tests rather than a sixth semantic category. **Diamond shape** already distinguishes them from rectangles; keeping them on-palette avoids an extra legend entry and matches the algorithmic convention in Figure 4 (Merge Sort).

Rationale for Five Categories: The five-category system balances cognitive load with visual distinctiveness. Five colours provide sufficient semantic granularity to capture the primary stages of most processes (input → structure → operation → intermediate → output) while remaining visually distinguishable and memorable. This number avoids both oversimplification (fewer categories would lose important distinctions) and cognitive overload (more categories would reduce rapid visual parsing). The colour choices (Red, Yellow, Green, Blue, Violet) follow a natural spectrum that aids memorability and intuitive association with process stages. Accessibility Considerations: The colour-coding system may present challenges for colorblind users. While the Framework allows customisation and alternative visual indicators (shapes, patterns, labels), the default five-colour scheme should be supplemented with text labels and, when possible, shape or pattern variations for accessibility. Node Shape Alternatives for Accessibility: Mermaid supports different node shapes that can serve as secondary identifiers independent of colour:

- **Ovals/stadium shapes** for triggers/inputs: `A([Input])`
- **Rectangles** for structures/objects: `B[Structure]`
- **Hexagons** for processing/operations: `C{{Operation}}`

- **Cylinders** for intermediates/states: `D[(State)]`
- **Trapezoids** for products/outputs: `E[/Output\]`

The present manuscript uses the suggested five-category fills in **Figures 1, 3, and 4**. **Figure 2** reproduces a **Class III** oxygen-sensing schematic from the empirical sequel companion (see §4.1) using that document's illustration palette (environmental signal, gene / protein roles, transcriptional outcome, and a distinct treatment of the degradation arm)—chosen to foreground **positive feedback** and alignment with circuit-topology exposition, not to assert a single universal colour law. Authors may optionally add Mermaid's auxiliary shapes (stadium inputs, cylindrical states, subroutine brackets for nested procedures, and so forth) when additional visual cues are warranted for accessibility. Future work should include systematic evaluation of colorblind accessibility and development of alternative visual encoding schemes. Customisation and Adaptation: The colour scheme is not rigidly enforced. Different processes may require different colour assignments based on domain-specific needs. Some GLMP and companion empirical figures adopt the **five-category Programming Framework palette** for pedagogy; others standardise **signal / gene / outcome** styles (as in topology illustrations for circuit-class papers). Older public snapshots may still show legacy palettes until refreshed. Other processes might benefit from domain-specific colour schemes that highlight particular aspects of the process (e.g., energy levels, time sequences, or hierarchical relationships). The suggested colour system can enable:

- Rapid visual parsing of process flow when consistently applied
- Cross-disciplinary comparison using consistent semantics (within a shared scheme)
- Pattern recognition across different domains
- Domain-specific customisation for specialised visualisation needs

3.3 LLM Prompt Engineering for Process Extraction

The core of the method uses LLMs to convert scoped text into an ordered set of process elements. A typical extraction prompt has the following structure:

1. Task statement:

   > You are a scientific process modeler. Given a textual description of a process, extract a sequence of steps, decisions, and flows. Optionally classify each element according to the Programming Framework's suggested colour system: Red (triggers/inputs), Yellow (structures/objects), Green (processing/operations), Blue (intermediates/states), Violet (products/outputs). Note that domain-specific processes may require custom colour assignments.

2. Output schema: Request a structured JSON-like or list-form output with elements such as:

   - id: step identifier
   - type: "trigger" | "structure" | "operation" | "intermediate" | "product"
   - label: concise description
   - inputs / outputs: optional fields for data or entities
   - colour: assigned colour category

3. Constraints:

   - No speculative steps beyond the text
   - Use consistent terminology for entities
   - Explicitly represent branching conditions
   - Maintain colour consistency throughout

4. Example:

   Provide 1–2 worked examples to anchor the model's behaviour, showing how different process types map to colour categories.

Given the scoped text and this prompt, the LLM produces a candidate structured representation with colour assignments. Prompt Sensitivity: Prompt design significantly influences extraction fidelity; this framework therefore treats prompts as first-class methodological artifacts. Small variations in prompt wording, example

selection, or constraint specification can yield substantially different outputs. The Framework addresses this through iterative refinement and validation workflows, acknowledging that prompt engineering is an ongoing process rather than a one-time configuration.

3.4 Mermaid Syntax and Structure

Mermaid is an open-source JavaScript-based diagramming and charting software that generates diagrams from text-based descriptions, created by Sveidqvist (2014). The project originated from a need to simplify diagram creation in documentation workflows and has since become widely adopted, with native support in GitHub, GitLab, Notion, Obsidian, and many other platforms (Sveidqvist, 2014; Wikipedia contributors, 2025). The Framework primarily uses Mermaid flowcharts with colour styling. Mermaid's text-based syntax enables version control, collaborative editing, and seamless integration into documentation systems.

GitHub's built-in diagram renderer applies a constrained Mermaid grammar: **parentheses or angle brackets inside edge labels** (`|...|`) must be wrapped as **quoted strings** (e.g. `|"degrades (normoxia)"|`); otherwise the lexer fails. Avoid HTML in edge labels; `<br>` inside **node** brackets is usually tolerated for line breaks.

Mermaid Markdown Code:

```
flowchart TD
A[Start/Input] --> B[Process Step]
B --> C{Decision Point}

C -->|Yes| D[Operation Path 1]
C -->|No| E[Operation Path 2]
D --> F[Intermediate State]
E --> F
F --> G[Final Output]
style A fill:#ff6b6b,color:#fff
style B fill:#51cf66,color:#fff
style C fill:#74c0fc,color:#fff
style D fill:#51cf66,color:#fff
style E fill:#51cf66,color:#fff
style F fill:#74c0fc,color:#fff
style G fill:#b197fc,color:#fff
```

Figure 1: Basic Programming Framework Structure. Color Legend: Red (#ff6b6b) = Triggers/Inputs, Yellow (#ffd43b) = Structures/Objects, Green (#51cf66) = Processing/Operations, Blue (#74c0fc) = Intermediates/States (including branch predicates in decision diamonds), Violet (#b197fc) = Products/Outputs. Note: This is one possible representation; more detailed versions can be generated by specifying greater granularity in the prompt.

The transformation from LLM output to Mermaid involves:

1. Node mapping: Assign each extracted step or decision to a Mermaid node with a short label
2. Edge construction: Translate the LLM's ordering and branching into --> or labeled edges (e.g., -->|Yes|)
3. Colour application: Apply the appropriate colour styling based on the Framework's five-category system (inline `mermaid` string); optionally mirror the same graph to a sibling UTF-8 `.mmd` file keyed by `mermaid_file` for HTML/`fetch`-based viewers.
4. Layout specification: Standardise on top-down (TD) or left-right (LR) layouts for consistency
5. Styling and grouping (optional): Use Mermaid's subgraphs or classes to group related steps or highlight categories

This transformation can be done by the same LLM (with a different prompt) or by deterministic code that maps a structured intermediate representation to Mermaid syntax with colour styling.

3.5 JSON Storage and Metadata

Each process diagram is stored as a JSON object that includes:

1. Core fields (required):
   - id: unique identifier
   - title: human-readable name
   - description: short textual description
   - mermaid: Mermaid source string (including colour styling)
   - mermaid_file (optional companion): filename of a UTF-8 `.mmd` file deployed next to the JSON (or under a documented static path) with the **same** graph as `mermaid` — enables HTML viewers to `fetch` diagram text without duplicating authoring logic
   - version: version number or timestamp
   - prompt_version: version identifier for the specific prompt template used (enables reproducibility)
   - llm_version: version of the LLM used for generation (e.g., "Gemini 2.0 Flash", "GPT-4")
2. Metadata fields (optional but recommended):
   - domain: e.g., biology, mathematics
   - category: subdomain or pathway family
   - source: reference to the originating paper, textbook, or URL
   - entities: key entities (genes, proteins, molecules, algorithms, etc.)
   - color_distribution: count of nodes by colour category (when using the §3.2 palette or a comparable tally)
   - palette / palette_note (optional): short label documenting a non–§3.2 scheme (e.g. empirical sequel topology colours)
   - annotations: notes or comments
   - created_by / reviewed_by: human or AI agents involved
3. Links:
   - Links to related diagrams (e.g., upstream or downstream processes)
   - Links to external systems (CopernicusAI briefings, metadata database entries, etc.)

This JSON format allows the diagrams to be treated as first-class data objects that can be indexed, searched, and cross-referenced. The schema is intentionally minimal and extensible, allowing domain-specific additions while maintaining core interoperability across disciplines.

The illustrative JSON record below (`beta-galactosidase-regulation`) matches the fields discussed above; **Figure 3** (§4.1) renders the companion `mermaid` graph at manuscript scale (`beta-galactosidase-regulation.mmd` alongside that JSON).

Example JSON Schema:


```
{
"id": "beta-galactosidase-regulation",
"title": "Beta-Galactosidase Regulation System",
"description": "Regulatory system controlling lactose metabolism in E. coli",
"mermaid": "(see Figure 3 / beta-galactosidase-regulation.mmd -- lacZYA schematic)",
"mermaid_file": "beta-galactosidase-regulation.mmd",
"version": "1.2",
"prompt_version": "v2.1",
"llm_version": "Gemini 2.0 Flash",
"domain": "biology",
"category": "gene_regulation",
"source": "Lac Operon: A Paradigm of Gene Regulation. Nature Reviews Genetics, 2005.",
"entities": ["LacI", "beta-galactosidase", "lactose", "cAMP", "CAP"],
"color_distribution": {
"red": 2,
"yellow": 3,
"green": 7,
```

```
"blue": 8,
"violet": 2
},
"annotations": "Validated against textbook description. Reviewed by domain expert.",
"created_by": "LLM (Gemini 2.0 Flash)",
"reviewed_by": "Domain Expert",
"links": {
"related_diagrams": ["lactose-transport", "glucose-metabolism"],
"external": {
"glmp": "https://huggingface.co/spaces/garywelz/glmp",
"source_paper": "https://doi.org/10.1038/nrg1234"
}
}
}
```


3.6 Iterative Refinement Workflow

The initial LLM-produced diagram is rarely perfect. The Framework therefore treats each diagram as a draft subject to iterative improvement:

1. AI Consensus Step (Optional but Recommended):
    - A second LLM (a "critic" model) reviews the first LLM's diagram against the source text
    - The critic model flags potential errors, missing steps, or inconsistencies
    - This reduces the burden on human reviewers by catching obvious errors before expert review
    - Example critic prompt: "Review this process diagram for accuracy. Compare it to the source text and identify any missing steps, incorrect relationships, or logical inconsistencies."
2. Review loop:
    - Human reviewer checks correctness against source
    - Reviewer flags errors (missing steps, incorrect branches, colour misassignments, oversimplifications)
    - Reviewer either edits the Mermaid directly or uses guided prompts to ask the LLM for revisions
3. Validation prompts:
    - "Compare this diagram to the source text and list any missing steps."
    - "Identify any logical inconsistencies between the diagram and the description."
    - "Verify colour assignments against the chosen scheme for each deployment (for example the suggested five-category palette in §3.2, or the empirical companion illustration conventions where Figure 2 is sourced)."
4. Versioning:
    - Each set of changes increments the version field
    - Past versions are retained for audit and comparison

Over time, this creates a library of increasingly accurate diagrams with traceable revision histories. The AI consensus step reduces human labor cost while maintaining quality through multi-model validation.

Validation methodology

The Framework employs a multi-stage validation process to ensure diagram accuracy:

1. Initial Validation:
    - **Who validates:** domain experts (for high-stakes scientific content) or knowledgeable reviewers (for educational content)
    - **Criteria for correctness:**
        - All steps from source text are represented

- Branching logic matches source description
- Colour assignments follow Framework conventions (or documented domain-specific customisations)
- No hallucinated steps beyond source material
- Logical flow is consistent (no unreachable nodes, proper entry and exit points)

2. Structured Validation Checklist:

   - Completeness: Are all key steps from the source represented?
   - Accuracy: Do the relationships and flows match the source description?
   - Consistency: Are colour assignments consistent throughout **within the palette chosen for that diagram**?
   - Clarity: Is the diagram readable and interpretable?

3. Agreement Metrics (Future Work):

   - Current implementation relies on single-reviewer validation
   - Future work should include inter-rater reliability studies
   - Multiple reviewers could independently validate diagrams, with agreement metrics (e.g., Cohen's kappa) calculated for correctness assessments

4. Stopping Conditions:

   - Validation continues until reviewer confirms diagram accuracy
   - For high-stakes content, multiple review cycles may be required
   - Version history tracks all changes, enabling rollback if errors are discovered later

5. Provenance Tracking:

   - Each diagram includes metadata about who created, reviewed, and validated it
   - Source references enable verification against original materials
   - Version history provides audit trail for all modifications

4. Applications Across Disciplines

The Programming Framework has been applied across multiple scientific domains. The most developed deployments are the Genome Logic Modelling Project (biology) and the Algorithms, Axiomatic Theories, and Proofs work in mathematics. The sections below develop biology (§4.1) and mathematics (§4.2); §4.3 compares the Framework to specialised domain tools.

4.1 Biological Processes: Genome Logic Modelling Project (GLMP)

The most extensive application to date is the Genome Logic Modelling Project (GLMP), which applies the Framework to map biological processes as text-based Mermaid flowcharts stored with metadata—**diagrams-as-code** that diff cleanly in version control and can sit beside analysis scripts, lab notes, and supplementary materials.

Complement to databases and quantitative models. GLMP charts are not substitutes for kinetic models, single-cell atlases, or predictive machine-learning systems; they make **conditional structure** and **branching logic** explicit—inputs, regulators, branch points, feedback, and candidate levers or readouts—before major experimental or computational commitments, as **reviewable hypothesis artefacts** rather than fitted simulations.

Curated wiring versus interpretive overlays. For bacterial gene regulation, RegulonDB (Salgado et al., 2024) foregrounds **who regulates whom** with strong entity completeness; textbook and LLM-assisted charts often add **explicit gate-style** questions (e.g. simultaneous relief of repression and favourable catabolite control for strong induction). These are **different abstraction levels**: database exports rarely include the same **Boolean-style** simultaneous conditions as pedagogical diamonds, and fusing all node types from incompatible ontologies can **obscure** branch logic. **Layered hybridization** works better: a

regulatory backbone from a trusted interaction resource; a **literature- or LLM-assisted overlay** for gates, feedback, and conditionality; then optional enzyme, reaction, and identifier audits once topology stabilises. **Disagreements** (e.g. *lacI* transcription explicit versus folded into a "repressor" node) signal where to refine.

Scope. Many processes across organisms and systems; categories include Central Dogma, metabolism, signalling, proteins, photosynthesis, DNA repair, and broad coverage of regulatory networks, metabolic pathways, and cellular processes.

Data sources. Textbooks, reviews, open-education resources, and primary literature—combined, in curated deployments, with cross-checks to database-backed facts where available.

Pipeline.

1. Select a process (e.g. a GLMP database entry, or a schematic from a companion methods / empirical paper).
2. Canonical description + references.
3. LLM pipeline → initial Mermaid + colours.
4. Refine with experts (and database anchors if used).
5. Store JSON in cloud.
6. Publish via interactive viewer.

Outcomes. A coherent styled collection of biological process diagrams; a public demonstration of the Framework in a complex domain; integration with the broader CopernicusAI Knowledge Engine.

Affordances for experiment-oriented reasoning (proposed, not evaluated). Reviewed GLMP maps can *support* qualitative **experiment design** and **perturbation-oriented discussion** (plausible levers; **structural downstream** reachability in the diagram, not quantitative prediction) and **stratification** by feedback density, loop content, and **source mixture**. Expert validation remains essential; **large diagrams** need **stronger prompting and iterative review** before being read as **publication-ready** summaries.

A companion methods draft (bioRxiv-oriented) contrasts literature-first, database-emphasis, and hybrid encodings of the *lac* operon for readers who want side-by-side figures.

Circuit-class companions. Empirical and theoretical companions that classify regulatory graphs sometimes publish **minimal topology figures** whose colours encode **roles in the illustration** (signal, gene product, transcriptional output; emphasis on degradation or activation edges) rather than the §3.2 teaching palette. That choice keeps figures aligned across a paper series; the Programming Framework contribution remains **Mermaid-as-source** and metadata interchangeability.

Example: **Class III — VHL–HIF oxygen sensing** (from the empirical sequel companion *Circuit class predicts virtual cell model accuracy*, topology illustrations §3.4). Under normoxia, VHL targets HIF1α for degradation; under hypoxia, HIF1α stabilises and activates VEGF, GLUT1, and glycolytic genes whose programme **feeds back** to reinforce the hypoxic state—**positive feedback** and **bistability** around an oxygen threshold. The companion cites benchmark context for genes in this motif (mean Pearson $r = 0.921$ for the VHL-associated Class III illustration panel—the highest among the Class III schematic examples reproduced there—alongside aggregate stratification showing Class III genes are systematically harder for grammar-blind virtual-cell models than Class I feed-forward genes).

Sample Diagram — Class III: VHL–HIF oxygen sensing (empirical sequel illustration palette)

Figure 2: VHL–HIF oxygen sensing as a **Class III** schematic (after the empirical sequel companion). Colours follow that publication's illustration convention—**not** the five-category §3.2 default used in Figures 1, 3, and 4. Companion HTML: https://storage.googleapis.com/regal-scholar-453620-r7-podcast-storage/mathematics-processes-database/empirical_sequel_draft.html — GLMP: https://huggingface.co/spaces/garywelz/glmp

Legend (this figure only): Blue — environmental signal (oxygen); teal — protein nodes (VHL E3 ligase, HIF1α); orange — transcriptional programme output (representative targets); dashed degradation arm — normoxic proteolytic targeting of HIF1α (emphasis stroke); solid arrows — transcriptional activation and **positive feedback** onto HIF1α.

Mermaid Markdown Code:

```
flowchart TD
    O2[Oxygen level] --> VHL[VHL E3 ligase]
    VHL -.->|"degrades (normoxia)"| HIF[HIF1α]
    HIF -->|"activates (hypoxia)"| Targets["VEGF, GLUT1<br>glycolytic genes"]
    Targets -->|"positive feedback"| HIF

    style O2 fill:#2E75B6,stroke:#1a5276,color:#fff
    style VHL fill:#1abc9c,stroke:#16a085,color:#fff
    style HIF fill:#1abc9c,stroke:#16a085,color:#fff
    style Targets fill:#f39c12,stroke:#e67e22,color:#fff

    linkStyle 1 stroke:#e74c3c,stroke-width:2px
```

Sample Diagram — *lac* operon / beta-galactosidase regulation (*E. coli*, Programming Framework palette)

Figure 3: Condensed *lac* regulatory schematic (§3.2 colours): lactose and glucose inputs; LacI-mediated operator control; cAMP–CAP catabolite regulation; polycistronic *lacZYA* expression; lactose hydrolysis and feedback via permease and metabolic signals. Full multi-encoding corpus entry: https://storage.googleapis.com/regal-scholar-453620-r7-podcast-storage/glmp-v2/viewer_demo/glmp-viewer-demo-v1-v6.html?process=ecoli_lac_operon&version=v1 — JSON schema illustration: §3.5 (`beta-galactosidase-regulation`).

Colour Legend: Red (Triggers/Inputs), Yellow (Structures/Objects), Green (Processing/Operations), Blue (Intermediates/States; branch predicates), Violet (Products/Outputs).

Mermaid Markdown Code:

```
flowchart TD
    LacExt[Lactose outside cell] --> ImpL[Lactose uptake LacY]
    GlcExt[Glucose outside cell] --> ImpG[Glucose uptake]
    ImpL --> LacIn[Intracellular lactose]
    ImpG --> GlcHi[High intracellular glucose]

    LacIn --> Qlac{Lactose present}
    Qlac -->|no| RepBlock[LacI blocks operator]
    Qlac -->|yes| PromOk[Promoter accessible]

    RepBlock --> TxBlocked[Transcription blocked]

    PromOk --> TxGate{Glucose low}
    GlcHi --> TxGate

    TxGate -->|no| TxWeak[Weak lacZYA transcription]
    TxGate -->|yes| TxStrong[Strong lacZYA transcription]

    TxWeak --> lacRNA[lacZYA mRNA]
    TxStrong --> lacRNA

    lacRNA --> TrZ[Translate LacZ]
    lacRNA --> TrY[Translate LacY]
    TrZ --> LacZ[Beta-galactosidase]
    TrY --> LacY[Lactose permease LacY]

    LacZ --> Hydro[Lactose hydrolysis]
    Hydro --> Sugars[Glucose and galactose]
```

```
Sugars --> Relax[Metabolic flux relieves lactose dependence]
Relax --> LacIn
LacY --> ImpL

TxBlocked --> NoProt[No lac enzymes expressed]

style LacExt fill:#ff6b6b,color:#fff
style GlcExt fill:#ff6b6b,color:#fff

style ImpL fill:#51cf66,color:#fff
style ImpG fill:#51cf66,color:#fff

style LacIn fill:#74c0fc,color:#fff
style GlcHi fill:#74c0fc,color:#fff
style PromOk fill:#74c0fc,color:#fff
style lacRNA fill:#74c0fc,color:#fff
style Relax fill:#74c0fc,color:#fff
style TxBlocked fill:#74c0fc,color:#fff

style Qlac fill:#74c0fc,color:#fff
style TxGate fill:#74c0fc,color:#fff

style RepBlock fill:#ffd43b,color:#000
style LacZ fill:#ffd43b,color:#000
style LacY fill:#ffd43b,color:#000

style TxWeak fill:#51cf66,color:#fff
style TxStrong fill:#51cf66,color:#fff
style TrZ fill:#51cf66,color:#fff
style TrY fill:#51cf66,color:#fff
style Hydro fill:#51cf66,color:#fff

style Sugars fill:#b197fc,color:#fff
style NoProt fill:#b197fc,color:#fff
```

4.2 Mathematical Processes

Mathematics routinely mixes **three kinds of objects** that are usually published in disconnected notations: **algorithms** (procedures), **axiomatic theories** (dependency among statements), and **proofs** (justifications). The Programming Framework deployment *Algorithms, Axiomatic Theories, and Proofs* treats all three as **labeled directed graphs** in Mermaid, generated from natural-language descriptions with LLMs, reviewed by humans, and stored as **versioned JSON** in the public **Mathematics Database**—a domain-specific extension; a companion manuscript (in preparation) documents the corpus and schema in full.

Algorithmic flowcharts in the mathematics corpus commonly use the Framework's **five-category palette** (§3.2) where authors adopt it for teaching-style figures: nodes are steps, decisions, or states; edges are sequential or conditional flow with explicit **loops and branching**. Corpus metadata records coarse structural indicators for comparison. Representative entries include the Sieve of Eratosthenes, Merge Sort, Dijkstra's algorithm, and the Euclidean algorithm.

Axiomatic dependency graphs represent logical **depends-on** structure among axioms, definitions, lemmas, theorems, and related objects—naturally read as **DAGs** (edges: "B relies on A," not temporal order). Colours encode **logical object type**. Examples include Euclid's *Elements*, Peano Arithmetic, ZFC, and algebraic theories; **depth from axioms** is an informal readout.

Proof graphs annotate steps with an **eight-role vocabulary**—source, assumption, construction, assertion, inference, **algorithm capsule**, contradiction, conclusion—so **embedded procedures** (constructions, diagonal routines, arithmetisation bookkeeping) appear as **explicit types**, supporting cross-proof comparison without claiming machine-checked correctness.

Proof assistants (Lean, Coq, Isabelle) target **verification** and proof terms; these diagrams target **human-facing layout** for inspection and teaching—**complementary** registers (companion draft contrasts a primes proof schematically).

At the level of **abstract graphs**, theory graphs are **acyclic**; algorithms may **cycle**; proof graphs are often **DAG-like** but may show **cyclic control** inside capsules. This is **interpretive** vocabulary only.

Corpus. The May 2026 manifest lists roughly **225** processes (mostly axiomatic theories, plus algorithms and proof graphs). Table and live previews: https://storage.googleapis.com/regal-scholar-453620-r7-podcast-storage/mathematics-processes-database/mathematics-database-table.html — https://huggingface.co/spaces/garywelz/programming_framework — Optional **AND-join** styling for conjunctive premises (primes pilot): https://storage.googleapis.com/regal-scholar-453620-r7-podcast-storage/mathematics-processes-database/proof-graphs/infinitely-many-primes-v2-demo.html

The Merge Sort example below illustrates a richer algorithmic flowchart than a minimal loop: recursion appears as subroutine-style brackets, splitting and merging are explicit operations (Green), pointers and interim lists are intermediates (Blue), partitions are structures (Yellow), and returns are outputs (Violet). The iterative merge loop deliberately cycles on the merged pair of lists to show typical control-flow density in real algorithms—without expanding every recursive descent as a duplicate subgraph.

```
flowchart TD
  IN["Input: sequence A with length n"] --> Q{"Length is 1 or 0?"}
  Q -->|yes| OUT1["Return A"]
  Q -->|no| MID["Compute mid equals floor(n/2)"]
  MID --> CUTL["L is prefix segment of A before mid"]
  MID --> CUTR["R is complementary suffix of A"]
  CUTL --> SL[[Recursive sort L]]
  CUTR --> SR[[Recursive sort R]]
  SL --> LS["L sorted"]
  SR --> RS["R sorted"]
  LS --> MG["Merge L_sorted with R_sorted"]
  RS --> MG
  MG --> INI["i,j,k to 0; empty output buffer Out"]
  INI --> WH{"Both halves have unmerged heads?"}
  WH -->|yes| CMP{"Left head not after right head?"}
  CMP -->|yes| APL["Take from L_sorted; advance i"]
  CMP -->|no| APR["Take from R_sorted; advance j"]
  APL --> WH
  APR --> WH
  WH -->|no| HC{"Remainder in L_sorted?"}
  HC -->|yes| FL["Append rest of L_sorted"]
  HC -->|no| HR{"Remainder in R_sorted?"}
  FL --> HR
  HR -->|yes| FR["Append rest of R_sorted"]
  HR -->|no| FIN["Build merged ordering"]
  FR --> FIN
  FIN --> OUT2["Return merged sorted sequence"]

  style IN fill:#ff6b6b,color:#fff
  style OUT1 fill:#b197fc,color:#fff
  style OUT2 fill:#b197fc,color:#fff
```

```
style CUTL fill:#ffd43b,color:#000
style CUTR fill:#ffd43b,color:#000
style LS fill:#74c0fc,color:#fff
style RS fill:#74c0fc,color:#fff
style SL fill:#ffd43b,color:#000
style SR fill:#ffd43b,color:#000
style Q fill:#74c0fc,color:#fff
style WH fill:#74c0fc,color:#fff
style CMP fill:#74c0fc,color:#fff
style HC fill:#74c0fc,color:#fff
style HR fill:#74c0fc,color:#fff
style MID fill:#51cf66,color:#fff
style MG fill:#51cf66,color:#fff
style INI fill:#51cf66,color:#fff
style APL fill:#51cf66,color:#fff
style APR fill:#51cf66,color:#fff
style FL fill:#51cf66,color:#fff
style FR fill:#51cf66,color:#fff
style FIN fill:#51cf66,color:#fff
```

Figure 4: Merge Sort. Recursive divide phase with subroutine-style calls, concatenated with an explicit iterative merge sweep (compare–append cycle and tail flushing). Larger composite layouts (multiple recursive levels unfolded, cost annotations) can be generated with richer prompts; the live mathematics-process database hosts additional variants.

4.3 Comparison with Existing Visualisation Tools

The Programming Framework addresses gaps in existing process visualisation tools across scientific domains. This section provides a consolidated comparison highlighting common themes and domain-specific considerations.

Domain-specific standards and tools

Domain	Typical standards	Representative tools	Primary focus
Biology	SBGN (Le Novère et al., 2009), BioPAX (Demir et al., 2010), SBML (Hucka et al., 2003)	CellDesigner (Funahashi et al., 2008), Cytoscape (Shannon et al., 2003), PathVisio, VANTED	Precise biochemical notation; quantitative modelling
Mathematics	No single standard covering algorithms, axiomatics, and proofs together	Coq, Isabelle, Lean (formal proof); MATLAB, Mathematica (computation)	Formal verification or numeric work; not lightweight unified diagrams
The Programming Framework	Mermaid Markdown (text-based)	LLM + Mermaid.js	Cross-disciplinary rapid visualisation; accessible process representation

Common limitations of existing tools:

Existing tools share recurring limitations: costly licences, steep learning curves, dependence on installed or cloud software, proprietary formats that resist portability, domain silos that hinder cross-disciplinary comparison, and interfaces or binary formats that raise access barriers.

Advantages of the Programming Framework's Mermaid-Based Approach:

1. Accessibility:

- Text-based format readable by humans without specialised software
- Can be edited in any text editor
- No software installation required for viewing (renders in GitHub, documentation systems, web browsers)

2. Version Control:
   - Native compatibility with Git and other version control systems
   - Enables collaborative editing and change tracking
   - Supports branching and merging workflows
3. Low Barrier to Entry:
   - Minimal learning curve compared to specialised tools
   - No licensing costs
   - Works on any platform with a web browser
4. Integration:
   - Easy integration into documentation, websites, and web applications
   - Can be embedded in Markdown documents, Jupyter notebooks, and documentation systems
   - JSON storage enables programmatic access and integration
5. Cross-Domain Consistency:
   - The same text-based format supports both biological (GLMP) and mathematical (Algorithms, Axiomatic Theories, and Proofs) deployments in one representational family
   - Enables comparison when diagram families are placed in the same tooling pipeline
   - Suggested colour semantics can facilitate pattern recognition when consistently applied
6. Rapid Prototyping:
   - Fast iteration cycles for exploring process representations
   - LLM-assisted generation reduces initial authoring time
   - Easy to modify and refine

Complementary Use Cases.

The Programming Framework is not intended to replace specialised tools for high-precision quantitative modelling, formal verification, or standards compliance. Instead, it serves complementary purposes:

- Educational contexts: quick visualisation for teaching and learning
- Rapid exploration: initial process mapping before detailed modelling
- Cross-disciplinary comparison: comparing processes across domains using consistent representation
- Documentation: embedding process diagrams in documentation and web content
- Collaborative workflows: version-controlled, text-based collaboration
- Integration: embedding process visualisations in knowledge engines and research platforms

For researchers requiring precise biochemical notation (SBGN), quantitative modelling, formal proof verification, or full UML notation, specialised tools remain the appropriate choice. The Programming Framework offers an accessible alternative for contexts where rapid visualisation, cross-domain comparison, and ease of use are priorities.

5. Results and Discussion

This section summarizes qualitative and quantitative observations from using the Programming Framework across multiple domains.

5.1 Process Coverage

The two most developed applications are:

1. **Biology (Genome Logic Modelling Project):** process diagrams spanning multiple pathway families and organisms, with a companion research paper in preparation.
2. **Mathematics (Algorithms, Axiomatic Theories, and Proofs):** algorithmic flowcharts, axiomatic dependency graphs, and proof graphs across multiple structural categories — see Section 4.2 and the live database — with a companion research paper in preparation.

This shows that the method is flexible enough to handle diverse content across scientific disciplines, given appropriate prompts and scoping.

5.2 Benefits Observed

Benefits observed across domains include: standardised representation via Mermaid, which is readable, widely supported, and amenable to version control; reduced authoring burden through LLM-assisted first-pass extraction with human validation; improved comparability through shared colour semantics and JSON metadata; cross-disciplinary insights when structurally similar processes from different fields are placed in the same representational space; and reusability through links to briefings, papers, web viewers, and programmatic access to diagram libraries. For GLMP, an additional benefit—in the sense of **affordances of the artefacts** rather than empirical outcomes—is that validated pathway maps can scaffold qualitative **experiment design**, **perturbation-oriented reasoning** about **diagram-level downstream structure**, and informal **comparison of processes by feedback and loop density**, especially when diagrams are **cross-checked** against curated regulatory wiring (see §4.1). For mathematics, the corpus places **algorithms**, **axiomatic DAGs**, and **proof graphs** (with **algorithm capsules**) in one pipeline—**complementary to proof assistants** for verification—and supports informal comparison of **acyclic**, **cyclic**, and **capsule-heavy** topologies (§4.2).

5.3 Limitations and Failure Modes

Despite its benefits, the Framework has clear limitations:

1. LLM hallucinations and shortcuts:
   - The model may infer steps not present in the source text
   - It may oversimplify or omit important branches when the source is ambiguous
   - Colour assignments may be inconsistent without careful prompt engineering
2. Granularity mismatch:
   - Without careful scoping, diagrams may be too coarse (missing key detail) or too fine-grained (overwhelming users)
   - Different domains may require different levels of detail
3. Domain-specific nuance:
   - Some fields require notations or conventions that Mermaid cannot fully express
   - For biology, Mermaid lacks the precision of SBGN (Le Novère et al., 2009) or the quantitative modelling capabilities of specialised tools like CellDesigner (Funahashi et al., 2008) and Cytoscape (Shannon et al., 2003)
   - Subtle mechanistic distinctions can be lost in generic flowchart form
   - Colour semantics may need domain-specific interpretation
   - The Framework is not a replacement for specialised tools when precision, quantitative modelling, or standards compliance are required
4. Human time cost for validation:
   - While authoring is faster, thorough validation still requires expert review
   - For high-stakes scientific use, domain experts must be involved
   - Colour assignment verification requires domain knowledge
5. Quantitative evaluation limitations:
   - The current work provides qualitative demonstrations across multiple domains but lacks systematic quantitative validation
   - No formal accuracy metrics comparing LLM-generated diagrams to expert-created ground truth
   - No inter-rater reliability studies for diagram correctness
   - Future work should include validation studies with domain experts and quantitative accuracy measurements
6. Format expressivity: Mermaid prioritises simplicity, human-readability, and version control over the full precision of specialist diagramming languages such as SBGN (see §3.4).

Mitigation strategies. The Framework addresses these limitations chiefly through:

1. Multi-checker workflows using multiple AI models for cross-validation
2. Human-in-the-loop review with domain experts
3. Iterative refinement cycles that improve accuracy over time
4. Version control and provenance tracking to maintain audit trails
5. Structured validation prompts that systematically check for common error patterns

These approaches reduce—but do not remove—the need for careful validation in high-stakes scientific applications.

5.4 Ethical Considerations

The use of LLMs for scientific process extraction raises several ethical considerations that must be addressed:

1. Accuracy and Responsibility:
   - LLM-generated diagrams may contain errors or hallucinations that could mislead researchers or students
   - The Framework mitigates this through human validation, but users must understand that diagrams require expert review before use in high-stakes contexts
   - Responsibility for scientific accuracy ultimately lies with human validators, not the AI system
2. Bias in Process Extraction:
   - LLMs may reflect biases present in training data, potentially emphasising certain process aspects while de-emphasising others
   - Domain-specific knowledge may be underrepresented in training data, leading to incomplete or culturally biased representations
   - Validation by domain experts from diverse backgrounds helps mitigate these concerns
3. Accessibility and Equity:
   - The Framework's text-based approach improves accessibility compared to proprietary tools
   - However, colour-coding may present challenges for colorblind users (addressed through customisation options)
   - Future work should include systematic evaluation of accessibility barriers
4. Intellectual Property:
   - Diagrams derived from copyrighted source materials must respect original authorship
   - The Framework's metadata system tracks sources, enabling proper attribution
   - Users should ensure compliance with copyright and fair use guidelines
5. Reproducibility:
   - LLM outputs can vary across model versions and prompt variations
   - The Framework addresses this through version control and prompt documentation (see Appendix A)
   - Users should document LLM versions and prompt configurations for reproducibility
6. Scientific Misinformation:
   - Incorrect process diagrams could propagate scientific misinformation
   - The Framework's validation requirements and version control help prevent this
   - Users should clearly indicate validation status and source materials

Recommendations

- Always validate LLM-generated diagrams with domain experts before publication or high-stakes use
- Maintain clear provenance and source attribution
- Document validation status and reviewer qualifications
- Consider accessibility needs when using colour-coding
- Use version control to enable error correction and rollback

6. Future Directions

Discipline-specific corpora for **biology** (GLMP) and **mathematics** (Algorithms, Axiomatic Theories, and Proofs) continue to grow as open, versioned resources.

Further work includes:

1. Stronger validation tooling: structural checks (unreachable nodes, entry/exit); domain rules where applicable.
2. Interactive editors: live Mermaid editing; AI-assisted suggestions; collaborative versioned workflows.
3. Formal semantics: optional maps to Petri nets, state machines, or other analyzable formalisms.
4. Knowledge-graph linking: entity resolution for nodes; multi-diagram queries.
5. Community libraries and domain extensions: public submission/review; specialised palettes and tool integration.

7. Conclusion

The Programming Framework offers a practical, reusable method for turning textual process descriptions into structured, computable diagrams using LLMs and Mermaid, enhanced by a suggested five-category colour-coding system that can be customised for specific domains. It has been applied most extensively in biology (via GLMP) and mathematics (via the Algorithms, Axiomatic Theories, and Proofs project), each with a companion draft research paper, and the need for careful human validation applies throughout. The framework's suggested colour-coding system provides a starting point for consistent visualisation, but recognises that different processes may benefit from custom colour schemes; curated GLMP charts increasingly adopt the five-category palette above, while legacy snapshots may retain older biology-specific encodings until refreshed. Rather than claiming to "solve" process modelling, the Framework is proposed as infrastructure: a starting point that others can adopt, critique, and extend. By providing a clear pipeline—from text to Mermaid to JSON, with iterative refinement and a suggested colour-coding system that can be customised—the Programming Framework lowers the barrier to creating and maintaining high-quality process representations across scientific disciplines. We invite researchers, educators, and practitioners to experiment with the method, contribute process examples, and help refine both the prompts and the underlying schemas. In the broader CopernicusAI Knowledge Engine, the Programming Framework plays a central role in making processes visible, comparable, and computable—an essential step toward more transparent and navigable scientific workflows. Availability: The Programming Framework methodology, examples, and tools are available at: https://huggingface.co/spaces/garywelz/programming_framework — A static HTML pack with in-browser Mermaid rendering and plain `.mmd` sources is produced beside the manuscript sources (see build script output: `html-manuscript/index.html` and `html-manuscript/diagrams/*.mmd`). Related Work: The Genome Logic Modelling Project (GLMP) demonstrates the Framework's application to biology: https://huggingface.co/spaces/garywelz/glmp

Acknowledgments

This work is part of the CopernicusAI Knowledge Engine project, which aims to create AI-powered tools for scientific research synthesis and knowledge discovery. The Programming Framework serves as a foundational methodological component enabling structured representation of processes across scientific disciplines. The author thanks the CUNY Graduate Center New Media Lab for institutional support. Large language models (including Claude by Anthropic and Gemini by Google) were used in the generation and iterative refinement of process diagrams described in this paper; all diagrams were reviewed and validated by the author. A preprint of this work is available on Zenodo.

References

Berners-Lee, T., Hendler, J., & Lassila, O. (2001). The semantic web. *Scientific American*, *284*(5), 34–43. https://doi.org/10.1038/scientificamerican0501-34

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., & Amodei, D. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, *33*, 1877–1901. https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfcb4967418bfb8ac142f64a-Abstract.html

Demir, E., Cary, M. P., Paley, S., Fukuda, K., Lemer, C., Vastrik, I., Wu, G., D'Eustachio, P., Schaefer, C., Luciano, J., Schacherer, F., Martinez-Flores, I., Hu, Z., Jimenez-Jacinto, V., Joshi-Tope, G., Kandasamy, K., Lopez-Fuentes, A. C., Mi, H., Pichler, E., Rodchenkov, I., Splendiani, A., Tkachev, S., Zucker, J., Gopalakrishnan, H., Ratjan, A., Ramirez, L., Iglesias, P. A., Deasy, J., Alexander, N., Ayanwola, T., ... Bader, G. D. (2010). The BioPAX community standard for pathway data sharing. *Nature Biotechnology*, *28*(9), 935–942. https://doi.org/10.1038/nbt.1666

Funahashi, A., Matsuoka, Y., Jouraku, A., Morohashi, M., Kikuchi, N., & Kitano, H. (2008). CellDesigner 3.5: A versatile modelling tool for biochemical networks. *Proceedings of the IEEE*, *96*(8), 1254–1265. https://doi.org/10.1109/JPROC.2008.925458

Gruber, T. R. (1993). A translation approach to portable ontology specifications. *Knowledge Acquisition*, *5*(2), 199–220. https://doi.org/10.1006/knac.1993.1008

Healey, C. G., & Enns, J. T. (2012). Attention and visual memory in visualisation and computer graphics. *IEEE Transactions on Visualisation and Computer Graphics*, *18*(7), 1170–1188. https://doi.org/10.1109/TVCG.2011.127

Hucka, M., Finney, A., Sauro, H. M., Bolouri, H., Doyle, J. C., Kitano, H., Arkin, A. P., Bornstein, B. J., Bray, D., Cornish-Bowden, A., Cuellar, A. A., Dronov, S., Gilles, E. D., Ginkel, M., Gor, V., Goryanin, I. I., Hedley, W. J., Hodgman, T. C., Hofmeyr, J., ... Wang, J. (2003). The systems biology markup language (SBML): A medium for representation and exchange of biochemical network models. *Bioinformatics*, *19*(4), 524–531. https://doi.org/10.1093/bioinformatics/btg015

Kong, N., Agrawala, M., & Hartmann, B. (2019). Perfopticon: Visual querying for performance in large-scale graph databases. *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*, 1–12. https://doi.org/10.1145/3290605.3300688

Larkin, J. H., & Simon, H. A. (1987). Why a diagram is (sometimes) worth ten thousand words. *Cognitive Science*, *11*(1), 65–100. https://doi.org/10.1016/S0364-0213(87)80026-5

Le Novère, N., Hucka, M., Mi, H., Moodie, S., Schreiber, F., Sorokin, A., Demir, E., Wegner, K., Aladjem, M. I., Wimalaratne, S. M., Bergman, F. T., Gauges, R., Ghazal, P., Kawaji, H., Li, L., Matsuoka, Y., Villéger, A., Boyd, S. E., Calzone, L., ... Kitano, H. (2009). The Systems Biology Graphical Notation. *Nature Biotechnology*, *27*(8), 735–741. https://doi.org/10.1038/nbt.1558

Liu, C., Wu, P., Li, C., & Zhu, J. (2020). Generating diagrams from natural language. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, 3637–3647. https://doi.org/10.18653/v1/2020.emnlp-main.295

Noy, N. F., & McGuinness, D. L. (2001). *Ontology development 101: A guide to creating your first ontology* (Stanford Knowledge Systems Laboratory Technical Report No. KSL-01-05). https://protege.stanford.edu/publications/ontolog

Object Management Group. (2017). Unified Modelling Language (UML), Version 2.5.1. https://www.omg.org/spec/UML/2.5.1

OpenAI. (2023). GPT-4 technical report. *arXiv*. https://arxiv.org/abs/2303.08774

Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C. L., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Askell, A., Welinder, P., Christiano, P., Leike, J., ... Ziegler, D. M. (2022). Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, *35*, 27730–27744.

Salgado, H., Gama-Castro, S., Lara, P., ... Collado-Vides, J. (2024). RegulonDB v12.0: A comprehensive resource of transcriptional regulation in *Escherichia coli* K-12. *Nucleic Acids Research*, *52*(D1), D255–D264. https://doi.org/10.1093/nar/gkad1072

Shannon, P., Markiel, A., Ozier, O., Baliga, N. S., Wang, J. T., Ramage, D., Amin, N., Schwikowski, B., & Ideker, T. (2003). Cytoscape: A software environment for integrated models of biomolecular interaction networks. *Genome Research*, *13*(11), 2498–2504. https://doi.org/10.1101/gr.1239303

Sveidqvist, K. (2014). Mermaid: Diagramming and charting software [GitHub repository]. https://github.com/mermaid-js/mermaid

Uschold, M., & Gruninger, M. (1996). Ontologies: Principles, methods and applications. *Knowledge Engineering Review*, *11*(2), 93–136. https://doi.org/10.1017/S0269888900007797

Welz, G. (2024). From inspiration to AI: Biology as visual programming. *Medium*. https://medium.com/@garywelz_47126/from inspiration-to-ai-biology-as-visual-programming-520ee523029a

Welz, G. (2024–2025a). The Programming Framework: A universal method for process analysis [Hugging Face Space]. https://huggingface.co/spaces/garywelz/programming_framework

Welz, G. (2024–2025b). GLMP: Genome Logic Modelling Project [Hugging Face Space]. https://huggingface.co/spaces/garywelz

White, J., Fu, Q., Hays, S., Sandborn, M., Olea, C., Gilbert, H., … Schmidt, D. C. (2023). A prompt pattern catalog to enhance prompt engineering with ChatGPT. *arXiv*. https://arxiv.org/abs/2302.11382

Wikipedia contributors. (2025). Mermaid (software). In *Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/wiki/Mermaid_(software)

Prior Work and Related Artifacts

Live demos and repositories are listed under *Availability* in the Conclusion. Manuscript figures are built as HTML + `.mmd` (browser Mermaid); https://mermaid.live is convenient for ad hoc paste-and-render checks.

Appendix A: Complete Prompt Template

Self-contained extraction prompt (colours match §3.2; hex codes repeated for copy-paste).

Task statement

You are a scientific process modeler. Given a textual description of a process, extract a sequence of steps, decisions, and flows. Classify each element according to the Programming Framework's suggested colour system.

Colour system

Use the five categories from §3.2 with these labels when you adopt that palette: **Red (#ff6b6b)** triggers/inputs; **Yellow (#ffd43b)** structures/objects; **Green (#51cf66)** processing/operations; **Blue (#74c0fc)** intermediates/states; **Violet (#b197fc)** products/outputs. Map **branch predicates** (decision diamonds, `{...}` in Mermaid) to **Blue**—same semantic category as conditional checkpoint states—not to a sixth colour. For companion-specific illustration palettes (signals, gene roles, outputs), document the mapping in prose or metadata instead of forcing these five labels.

Output format (JSON)

```
{
  "steps": [
    {
      "id": "step_1",
      "type": "trigger|structure|operation|intermediate|product",
      "label": "Concise description",
      "color": "red|yellow|green|blue|violet",
      "inputs": ["entity1", "entity2"],
      "outputs": ["entity3"]
    }
  ],
  "edges": [
```

```
    {
      "from": "step_1",
      "to": "step_2",
      "label": "optional edge label"
    }
  ]
}
```

Constraints

- No speculative steps beyond the text
- Use consistent terminology for entities
- Explicitly represent branching conditions
- Maintain colour consistency throughout

Example

Input: "When lactose is present, the Lac repressor releases from the operator, allowing RNA polymerase to bind and transcribe the lac operon genes."

Output:

```
{
  "steps": [
    {"id": "s1", "type": "trigger", "label": "Lactose Present", "color": "red"},
    {"id": "s2", "type": "operation", "label": "Repressor Release", "color": "green"},
    {"id": "s3", "type": "structure", "label": "RNA Polymerase", "color": "yellow"},
    {"id": "s4", "type": "operation", "label": "Transcription", "color": "green"},
    {"id": "s5", "type": "product", "label": "lac Operon mRNA", "color": "violet"}
  ],
  "edges": [
    {"from": "s1", "to": "s2"},
    {"from": "s2", "to": "s3"},
    {"from": "s3", "to": "s4"},
    {"from": "s4", "to": "s5"}
  ]
}
```

Applying this template with an LLM

Send **Task statement** through **Example** output, then paste the target description in a **follow-up message** (or append after a closing "Analyse this text:" line in scripts). Prompt wording is sensitive (§3.3).